NAME :- Manas Singh

Roll No. :- 23010301 7.

GitHub :- https://github.com/singh-Manas-Web/cloud-kd-parser

What I learnt :-

This project made me learn the actual use of cloud concepts like object storage, containers, Kubernetes and autoscaling all together on my own laptop.

I learnt how knowledge distillation works in practice. A big model (phi 3.5) labelled 3000 commands and a much smaller model (Phythia-70M) learned to copy its answers. I got that a 70-million parameters model could reach 100% valid ~~Jeso~~ JSON and 47% full-frame accuracy, while teacher itself got 52%. I also understood why only sequence-level distillation was possible here, Ollama gives back only text, and the two models use different vocabularies, so their probabilities cannot be compared.

On the Kubernetes side, the readiness probe finally made sense to me. During the experiment the new Pod stayed out of the service endpoints for about 19 secs., until the model was loaded, and only then started recieving traffic. I also saw HPA scale from 1 to 2 replicas under load and back to 1 afterwards and rolling update from v1 to v2 and back while requests kept coming, with only 1 failed out 929.

I also learnt the basics of CI/CD properly. My pipeline lints the code, runs the tests, evaluates the model from Github release, and blocks it if JSON validity is below 95%. Seeing the untrained model get stopped at the quality gate was a good moment.

Scanned with OKEN Scanner

## Challenges I faced & how I solved them :-

1. Setting up on Windows. I had to install WSL2, Ubuntu and Docker Desktop from scratch. Docker first gave permission denied error, which I solved by adding my user to docker group.
2. kubectl could not reach the cluster. K3d wrote the cluster address as host.docker.internal, which did not work from WSL. This was fixed by giving the cluster a fixed address (127.0.0.1:6550) in k3d.yaml and recreating it.
3. Python version :- Ubuntu came with python 3.14, which was too new for PyTorch 2.5.1. I installed python3.11 separately using uv.
4. Unstable Internet : pip installs and github release uploads kept failing with "connection reset by peer". I added more retries to pip and lowered MTU of WSL network, after which the uploads worked.
5. CI secret scan failing. gitleaks crashed on my very first push because there was no earlier commit to compare with. I replaced it with gitleaks command-line tool scanning the whole history.
6. Limited Hardware : With only 8 GB of RAM, I had to be careful about running Ollama, Docker and training at the same time. ~~Otto~~ luckily Ollama used my GTX 1650 GPU, so labelling took 2 hours instead of 5-6 hours it would have taken on CPU.

~~Time and Health :-~~

## findings and observations :-

⇒ The teacher's output quality decides the student's ceiling. 117 teacher replies broke the slot rules and had to be removed, and the student repeated some of the teacher's mistakes.

Scanned with OKEN Scanner

⇒ More training is not always better. V2, with one more epoch, was slightly worse than V1 (99% v/s 100% validity) which looks like overfitting.

⇒ The service ran comfortably under the limits; peak memory was 573 MiB of 1 GiB and p95 latency was 2.1s against 5s target. But both pods used their full 1 CPU, so CPU was the real bottleneck.

⇒ ~~But cost can~~ since everything ran on one laptop, the cluster cannot show real fault tolerance. If the laptop fails, everything fails.

## What could have been better:

⇒ I would keep a small validation split to stop training at best epoch instead of guessing.

⇒ I would make model faster and smaller, for example with quantisation, so each Pod could handle more requests.

⇒ I would test the full "fresh clone and make up" flow earlier instead of at the end.

## Conclusion:

⇒ This project connect many studied topics. The most useful part was facing real problems (network failure, permissions and version mismatches) and fixing them step by step.

Scanned with OKEN Scanner